

Pseudo-classes e Estado de Elementos no CSS

Guia Técnico e Objetivo — Conceitos, Sintaxe e Aplicações

Curso: HTML e CSS (Otávio Miranda - Aula 18) Foco: Seletores Reativos e Estados de Interface

Na criação de interfaces web modernas, os elementos HTML não são estáticos; eles reagem dinamicamente às interações do usuário (como passar o mouse, clicar, focar um campo) e aos seus próprios estados internos (como estar marcado, desabilitado ou visitado). As **pseudo-classes CSS** permitem aplicar estilos específicos a um elemento com base nesses estados, sem utilizar JavaScript.

1. Conceito Fundamental de Pseudo-classes

- **Definição:** Palavras-chave adicionadas aos seletores CSS (precedidas por dois-pontos `:`) que identificam um estado especial do elemento selecionado.
- **Sintaxe Geral:** `seletor:pseudo-classe { propriedade: valor; }`
- **Finalidade:** Adicionar reatividade visual e dinamismo à interface de forma performática e nativa no CSS.

2. Interação Visual com `:hover` e Suavização com `transition`

- **Pseudo-classe `:hover`:** Disparada quando o ponteiro do mouse se posiciona sobre o elemento (títulos, botões, links, divs).
- **Propriedade `transition`:** Declarada no estilo base do elemento para suavizar a alternância de propriedades (como cores e tamanhos) ao acionar o `:hover`. Sintaxe: `transition: propriedade tempo tipo;`

```
/* Estilo Base com Transição Suave */
.titulo-destaque {
  background-color: #3b82f6;
  color: #ffffff;
  transition: all 300ms ease-in-out;
}

/* Estado de Hover (Mouse Sobre) */
.titulo-destaque:hover {
  background-color: #1d4ed8;
}
```

3. Ciclo de Vida e Estados de Links (`:link`, `:visited`, `:hover`, `:active`)

A estilização de hiperlinks (`<a>`) segue uma ordem de precedência importante para o comportamento correto na navegação:

- **`:link`:** Seleciona links que ainda *não foram visitados* pelo usuário.
- **`:visited`:** Seleciona links que *já foram visitados* anteriormente no navegador.
- **`:hover`:** Estiliza o link no momento em que o mouse está posicionado sobre ele.
- **`:active`:** Seleciona o elemento no exato instante em que é clicado (com o botão do mouse pressionado).

```
a:link { color: #2563eb; } /* Link não visitado */
a:visited { color: #7c3aed; } /* Link visitado */
a:hover { color: #dc2626; text-decoration: underline; } /* Mouse sobre */
a:active { color: #059669; } /* Clique ativo */
```

4. Estados e Validação em Formulários (`:focus` , `:disabled` , `:enabled` , `:required`)

- `:focus` : Ativado quando o campo recebe o foco de digitação (via clique ou tecla Tab). Essencial para destacar o campo e melhorar a acessibilidade com `box-shadow` ou `border` .
- `:disabled` : Seleciona elementos desabilitados via atributo `disabled` , permitindo alterar o fundo e indicar o bloqueio com `cursor: not-allowed;` .
- `:enabled` : Seleciona campos ativos e prontos para interação.
- `:required` : Seleciona campos configurados com o atributo HTML `required` .

```
/* Destaque ao Focar no Campo */
input:focus {
  outline: none;
  border: 2px solid #2563eb;
  box-shadow: 0 0 5px rgba(37, 99, 235, 0.4);
}

/* Campo Desabilitado */
input:disabled {
  background-color: #e2e8f0;
  cursor: not-allowed;
}
```

5. Alternância de Estado com `:checked` e Combinadores Irmãos

- `:checked` : Aplica-se a seletores de formulário (checkbox ou radio) quando marcados.
- **Lógica de Interface sem JavaScript:** Combinando `:checked` com o seletor irmão adjacente (`+`) ou irmão geral (`~`), é possível alterar o estilo de elementos vizinhos (ex.: abrir menus retráteis ou exibir mensagens ao marcar uma caixa).

```
/* Altera o parágrafo irmão adjacente quando o checkbox está marcado */
input[type="checkbox"]:checked + p {
  background-color: #dcfce7;
  color: #166534;
  font-weight: bold;
}
```

6. Seletores Estruturais Posicionais (`:first-child` , `:last-child`)

- `:first-child` : Seleciona o elemento que for o primeiro filho de seu elemento pai.
- `:last-child` : Seleciona o elemento que for o último filho de seu elemento pai.
- **Aplicação Típica:** Estilização diferenciada para o topo ou base de listas, tabelas e coleções de elementos.

7. Quadro Comparativo de Pseudo-classes

Pseudo-classe	Estado / Gatilho	Caso de Uso Prático
<code>:hover</code>	Mouse sobre o elemento	Feedback de interatividade em botões e links
<code>:focus</code>	Elemento focado (clique/Tab)	Destaque de campos e acessibilidade
<code>:active</code>	Elemento sendo pressionado	Simulação de efeito de clique em botões
<code>:link</code> / <code>:visited</code>	Estado de navegação do link	Diferenciar páginas visitadas e não visitadas
<code>:disabled</code> / <code>:required</code>	Atributos de formulário	Identificar visualmente campos bloqueados ou obrigatórios
<code>:checked</code>	Checkbox/Radio marcado	Controle de visibilidade/estilo de elementos adjacentes
<code>:first-child</code> / <code>:last-child</code>	Primeiro ou último filho do pai	Ajuste de bordas e fundos no início e fim de listas

Prática: Exercício de Consolidação

Enunciado do Desafio:

Construa uma interface web de painel interativo contendo um formulário de cadastro e uma lista de tarefas estilizada, utilizando exclusivamente HTML e CSS puro (sem qualquer uso de JavaScript).

Resultado Esperado e Requisitos de Interface:

- **Comportamento dos Links:** Os links do painel devem indicar se foram visitados ou não, transicionar de cor ao passar o mouse e alterar o estilo no momento do clique.
- **Campos do Formulário:** O formulário deve apresentar campos normais, um campo obrigatório e um campo desabilitado. Ao receber foco, os campos ativos devem exibir borda destacada e sombra suave. O campo desabilitado deve indicar impossibilidade de uso alterando a cor e o ponteiro do cursor.
- **Interatividade por Estado:** O formulário deve conter um checkbox. Ao ser marcado, um bloco de mensagem posicionado logo após ele deve mudar automaticamente sua cor de fundo e formatação de texto.
- **Estilização de Lista:** Na lista de tarefas, o primeiro item deve possuir destaque na borda superior e o último item deve ter estilização diferenciada no rodapé.
- **Suavidade Visual:** As mudanças de estado de hover e foco devem utilizar transições graduais.

Observação para o Estudante: A estrutura das tags HTML, a escolha das propriedades CSS e a implementação do "como fazer" para atingir o resultado acima são de total responsabilidade do estudante.